SAHAYAK

STATIC SIGN COLLECTION, TRAINING & DEPLOYMENT

A repeatable step-by-step guide for adding one new static sign while preserving previous signs.

YOUR LOCAL PROJECT
C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training

THE SAFE WORKFLOW

1. Back up the current dataset and active model.
2. Collect the new static sign into the existing CSV.
3. Verify labels and sample counts.
4. Train a candidate model - do not promote immediately.
5. Run tests and check candidate labels.
6. Promote locally, test the camera, then publish the model.

GOLDEN RULE
Never train using only the new sign. The candidate must use the existing static dataset plus the new samples. Promote only after old and new labels have been verified.

STEP 1 - Open the project and back up

Open the existing SAHAYAK training folder in VS Code. These are PowerShell commands. Do not replace the working project with a fresh ZIP.

```
Set-Location "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
.\.venv\Scripts\Activate.ps1
python --version
```

Confirm the prompt shows (.venv). Avoid reinstalling working packages.

1A. Back up the dataset and active model

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path ".\backups" "static_batch_$stamp"
New-Item -ItemType Directory -Path $backup -Force | Out-Null
$files = @(
  ".\data\landmarks_2hand.csv",
  ".\model\sahayak_twohand.joblib",
  ".\model\labels_twohand.json",
  ".\continuous_gesture_db.db"
)
foreach ($file in $files) { if (Test-Path $file) { Copy-Item $file $backup -Force } }
Write-Host "Backup saved to: $backup"
Get-ChildItem $backup | Select-Object Name, Length
```

KEEP DATA SOURCES SEPARATE
Static training reads data\landmarks_2hand.csv. The empty/root gesture_db.db is not the static training source shown by this train_model.py. Do not delete either database.

1B. Confirm current CSV counts

```
@'
import csv
from collections import Counter
from pathlib import Path
p=Path("data/landmarks_2hand.csv")
with p.open(newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("CSV rows:",len(rows))
for label,count in sorted(Counter(r["label"].strip().upper() for r in rows).items()):
    print(f"{label}: {count}")
'@ | python -
```

STEP 2 - Collect one static sign

A static sign is identifiable from a stable hand pose rather than a required movement path. A two-hand pose can still be static. Confirm the intended ISL form with a reliable reference or fluent signer.

```
python collect_data.py
```

Follow the prompts and use one consistent uppercase label, for example PLEASE. Keep the hands clearly visible and capture varied attempts under natural camera position and lighting.

> **DO NOT REPLACE THE CSV**
> The collector should append rows to data\landmarks_2hand.csv. If the file is replaced, becomes empty, or old labels disappear, stop and restore the backup before training.

2A. Check the full set of labels and counts

```
@'
import csv
from collections import Counter
from pathlib import Path
with Path("data/landmarks_2hand.csv").open(newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
for label,count in sorted(Counter(r["label"].strip().upper() for r in rows).items()):
    print(f"{label}: {count}")
'@ | python -
```

2B. Minimum checks

- Header must contain label plus v0 through v125 (126 features).
- Labels must be non-empty and consistently named; do not mix spaces/underscores or case.
- The current trainer requires at least 10 samples per class; collect varied, non-duplicate samples.
- Review unexpected labels. FINE appeared in the CSV earlier; include it only if it is an intended class.

STEP 3 - Train a candidate only

Run the trainer without --promote first. The candidate-first version leaves the active model unchanged by default.

```
python train_model.py
```

The trainer should create these candidate files:

```
model\sahayak_twohand_candidate.joblib
model\labels_twohand_candidate.json
```

3A. Check candidate labels

```
Get-Content .\model\labels_twohand_candidate.json
```

All prior intended static labels plus the new label must be present. Stop if an old label is missing or an unexpected class appears.

3B. Check input shape and counts

```
@'
import csv, joblib
from collections import Counter
from pathlib import Path
with Path("data/landmarks_2hand.csv").open(newline="",encoding="utf-8-sig") as f:
    counts=Counter(r["label"].strip().upper() for r in csv.DictReader(f))
m=joblib.load("model/sahayak_twohand_candidate.joblib")
print("CSV counts:",dict(sorted(counts.items())))
print("Candidate classes:",sorted(map(str,m.classes_)))
print("Input features:",m.n_features_in_)
'@ | python -
```

Expected input size: 126. Save the report for your training notes.

> **100% HOLDOUT IS NOT PROOF**
> A random row split can place very similar samples from one recording session in both train and test. Test with fresh camera attempts and, ideally, different recording sessions or people.

STEP 4 - Test, then promote locally

4A. Run tests

```
python -m unittest test_sahayak.py
```

These tests check features and coordinate handling; they do not by themselves prove classifier accuracy.

4B. Review before promotion

- Candidate labels contain the new class and every old class intended to remain.
- CSV class counts are sensible and unexpected labels have been reviewed.
- The candidate accepts 126 features and the report has no obvious class-specific failure.
- You have fresh-camera trials for the new sign and the old signs.

4C. Promote when acceptable

```
python train_model.py --promote
```

This backs up the previous static model and labels under model\backups\, then replaces the active static model and labels. It should not change the continuous model or databases.

4D. Run and test the local app

```
python web_app.py
```

Open the local URL printed in the terminal. Test every old static sign and the new one. If old signs become unreliable, stop and restore the timestamped backup. Do not deploy yet.

Quick rollback

```
# Replace <timestamp> with the backup folder name.
Copy-Item ".\model\backups\<timestamp>\sahayak_twohand.joblib" ".\model\sahayak_twohand.joblib" -Force
Copy-Item ".\model\backups\<timestamp>\labels_twohand.json" ".\model\labels_twohand.json" -Force
```

STEP 5 - Publish only the tested static model

Only after local camera testing, copy the promoted model into the deployment project. Do not upload the candidate CSV or databases just to deploy recognition.

5A. Files and destinations

Local file	GitHub destination
model\sahayak_twohand.joblib	SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib
model\labels_twohand.json	SAHAYAK-Vercel-Deployment/backend/labels_twohand.json only if metadata needs updating

5B. Upload through GitHub

- Open https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL/tree/main/SAHAYAK-Vercel-Deployment/backend/model
- Choose Add file > Upload files and select the promoted sahayak_twohand.joblib. Keep the exact filename.

REUSABLE CHECKLIST & TROUBLESHOOTING

Use this checklist each time you add a static sign. The promotion command also backs up the active model and labels.

- [ ] Backed up data\landmarks_2hand.csv and active model/labels.
- [ ] Collected samples using collect_data.py with one consistent label.
- [ ] CSV counts include old classes plus the new class; unexpected labels reviewed.
- [ ] python train_model.py created a candidate without changing the active model.
- [ ] Candidate labels and 126-feature shape verified.